Discovering Knowledge in Data

Daniel T. Larose, Ph.D.

Chapter 6
Preparing to model the data

Prepared by James Steck and Eric Flores

Supervised vs Unsupervised Methods

- Data mining methods may be categorized as either *supervised* or *unsupervised*
- In *unsupervised methods*, there is no specific target variable
- The most common unsupervised method is *clustering* (Chapters 10 and 11)
  - For example, political consultant might use clustering to uncover voter profiles based on income, gender, race, etc., for fund-rising and advertising purposes
  - When used for market basket analysis (which products are bought together), *association mining* is also considered as a *unsupervised method*, as there is target variable (Chapter 12)
- Most data mining methods are *supervised*
  - There is a particular pre-specified target variable
  - The algorithm is given many examples where the value of the target variable is provided
  - The algorithm learns which values of the target variable are associated to which values of the predictor variables
  - Regression, from Chapter 5, is a supervised method
  - All classification methods from Chapters 7 to 9 (decision trees, neural networks, k-nearest neighbors) are supervised methods too
- Important: Supervised and Unsupervised are just data mining terms
  - Unsupervised methods do not mean that they require no human involvement!

Statistical methodology and data mining methodology

- Chapters 4 and 5 introduced many statistical methods for performing inference
- Statistical methods and data mining differ in two ways
  1. Applying statistical inference using the huge sample sizes tends to result in statistical significance, even when the results are of no practical significance
  2. In statistical methodology, the data analyst has an *a priori* hypothesis in mind
     - Data mining procedures usually do not have an *a priori* hypothesis, instead freely trolling through the data for actionable results

Cross-validation

- Unless properly conducted, data mining can return phantom spurious results due to random variation rather than real effects
- This data dredging can be avoided through *cross-validation*
- Cross-validations ensures that results uncovered in an analysis are generalizable to an independent, unseen, data set
- The most common methods are *twofold cross-validation* and *k-fold cross-validation*
- In twofold cross-validation, the data are partitioned, using random assignment, into a *training data set* and a *test data set*
  - The only systematic difference between the training data set and the test data set is that the training data includes the target variable and the test data does not; the training dataset will be *preclassified*

Cross-validation *(cont'd)*

- The training set is incomplete – it does not include new/future data
- The algorithm must not memorize and blindly apply patterns from training set into new/future data
  - Example: If all customers named "David" in the training set fall in the same Income bracket, we don't want to algorithm to assign Income bracket based on the "David" name
  - Such a pattern is a spurious artifact of the training set and needs to be verified before deployment
- The next step in supervised data mining is to examine how the data model performs in the test set of the data
  - The target variable of the test set is hidden temporarily, and classification is performed according to the predictor variables only
  - The efficacy of the classification are evaluated by comparing the predicted values against the true values of the target variable
  - The provisional data mining model is adjusted to minimize the error on the test set

Cross-validation *(cont'd)*

- Model evaluation for future data can be performed using techniques covered in Chapter 14
- Cross-validation guards against spurious results because it is highly unlikely that the same random variation is found in both the training and test set
- But the data analyst must ensure that the training and test sets are indeed independent, by *validating the partition*.
- Validate the partition into training and test sets by graphical and statistical comparison
  - We might find that a significantly higher proportion of positive values of an important flag variable were assigned to the training set – this would bias the results and hurt the prediction/classification
- Table 6.1 shows suggested hypothesis test for validating different types of target variables

Table 6.1

Type of Target Variable	Test from Chapter 5
Continuous	Two-sample t-test for difference in means
Flag	Two-sample Z test for difference in proportions
Multinomial	Test for homogeneity of proportions

Cross-validation *(cont'd)*

- In k-fold cross validation, the data is partitioned into k subsets or folds
- The model is then built using the data from k – 1 subsets, using the kth fold as the test set
- This is repeated k times, always using a different fold as the test subset, until we have k different models
- The results from the k models are then combined using averaging or voting
- A popular choice for k is 10
- A benefit of using k-fold cross-validation is that each record appears in the test set exactly once; a drawback is that the requisite validation task is made more difficult.

Cross-validation (*cont'd*)

- In summary
  1. *Partition* the available data into a *training set* and a *test set*. Validate the partition.
  2. Build a data mining model using the training set data.
  3. Evaluate the data mining model using the test set data.

Overfitting

- Usually, the accuracy of model is not as high on the test set as it is on the training set
  - This might be caused by *overfitting* on the training set
- Overfitting occurs when the model tries to fit every possible trend/structure in the training set, like the spurious data issues mentioned above
- There is a need to balance the model complexity (resulting in high accuracy in training set) and generalizability to the test/validation sets
  - Increasing complexity leads to degradataion of generalizability of the model to the test set, as shown in Figure 6.1 (next slide)
- Per Figure 6.1, as the model begins to grown in complexity, the error rates for both training sets starts to fall
- As the model complexity increases, the error in the training set continuous to fall as the error in the test set starts to increase
  - This is caused because the model has memorized the training set rather than leaving room for generalization to unseen data

Overfitting *(cont'd)*

- The optimal model complexity is the point where the minimal error rate on the test set is located
- Complexity greater than this is considered overfitting
- Less than this is considered underfitting

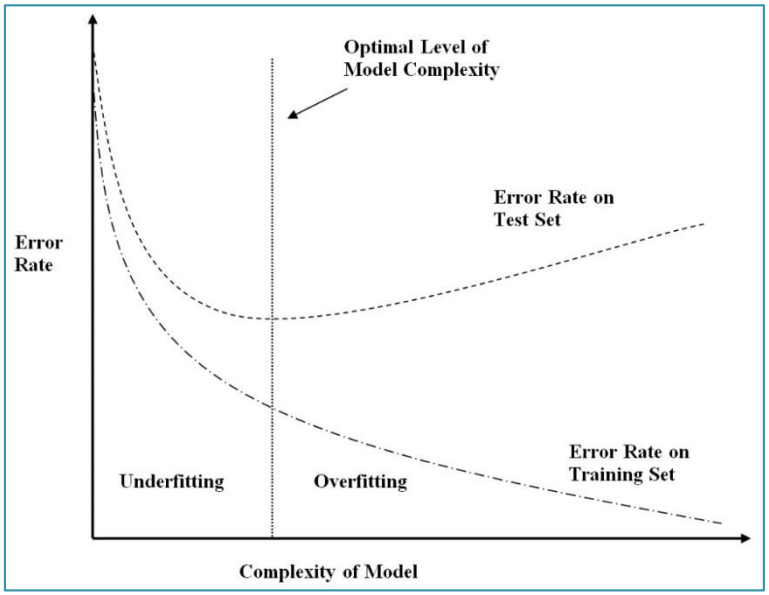

Figure 6.1

Bias-Variance trade-off

- Example: Building the optimal curve (or line) that separates dark gray points from light gray points in Figure 6.2
  - A straight line has low complexity, but some classification errors
- In Figure 6.3 we have reduced the error to zero, but at the cost of complexity
- One might be tempted to use the more accurate model
  - However, we should be careful not to depend on the idiosyncrasies of the training set

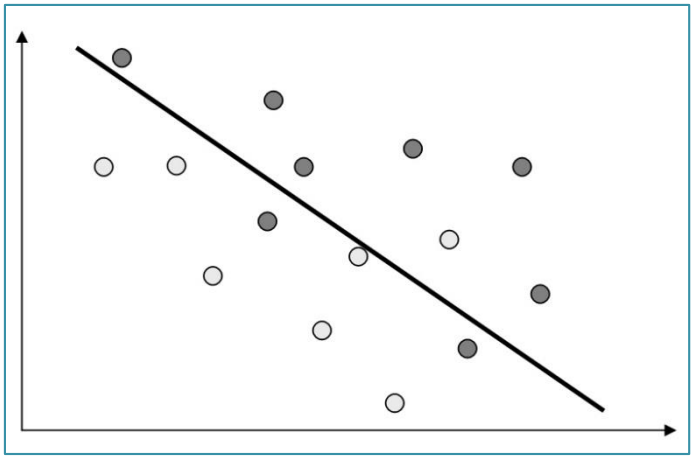

Figure 6.2

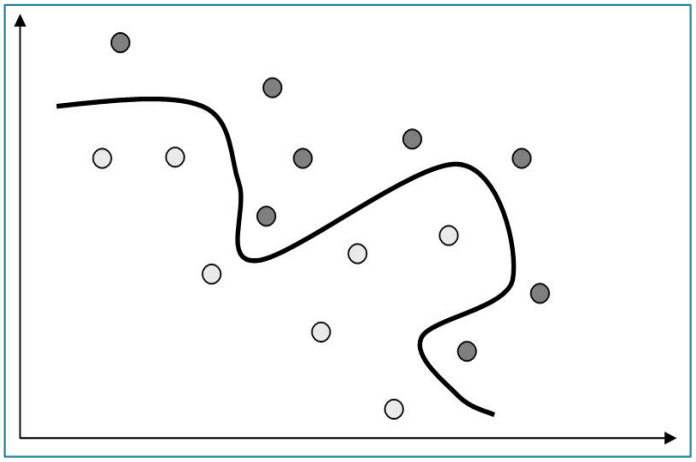

Figure 6.3

Bias-Variance trade-off (*cont'd*)

- Suppose that we add more data points to the scatter plot, as per Figure 6.4
- In this case, the low-complexity separator need not much change
  - This means that this separator has *low variance*
- But the high-complexity separator, the curvy line, must alter considerably to maintain its low error rate
  - This high degree of change indicates that this separator has *high variance*

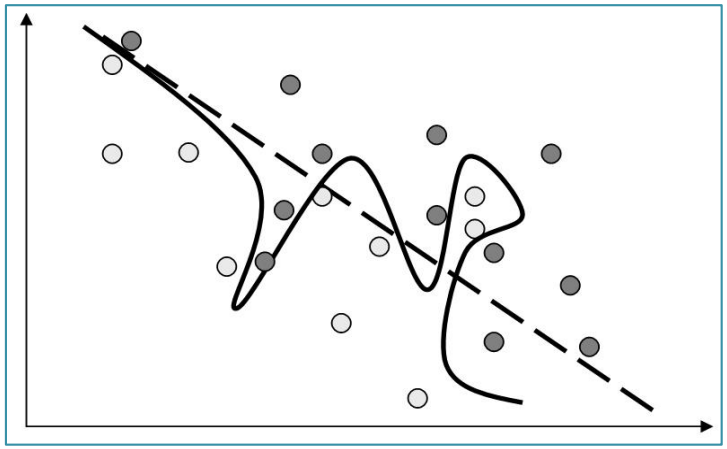

Figure 6.4

Bias-Variance trade-off *(cont'd)*

- Even thought the high-complexity separator has *low bias* (low error rate on the training set), it has *high variance*
- And, even thought the low-complexity model has a high *bias*, it has low *variance*
- This is known as the *bias-variance trade-off*
  - This is another way of describing the overfitting/underfitting dilemma from the prior section
  - As model complexity increases, the bias of the training set decreases, but the variance increases
  - The goal is to construct a model in which neither bias nor variance is too high
- Example: A common evaluation for accuracy of models for continuous target variables is mean-squared error (MSE)
  - Why is MSE such a good evaluative measure?
  - Because it combines both bias and variance
  - MSE is a function of the estimation error (SSE) and the model complexity (degrees of freedom)
  - It can be shown (Hand et al. [1]) that the MSE can be partitioned using the following equation: $MSE=variance+bias^2$

Balancing the training data

- Balancing is recommended on classification models when one target variable class has much lower frequency than the other classes
  - The algorithm have a chance to learn about all types of records, not just those with high frequency
- Example: In fraud classification model with 100,000 transactions, only 1000 are fraudulent
  - The model could achieve 99% accuracy by labeling all transactions as "non-fraudulent" – this behavior is not desired
  - Instead, balance should be performed so that relative frequency of fraudulent transactions is increased
- There are two ways of doing balancing
  1. Resample a number of fraudulent recods
  2. Set aside a number of non-fraudulent records

Balancing the training data *(cont'd)*

- *Resampling* refers to sampling at random with replacement from a data set
- Example: Use Resampling so that the fraudulent records represent 25% of the balanced training set, rather than 1%
  - Solution: Add 32,000 resampled fraudulent records so that we had 33,000 fraudulent records, out of a total of 100,000+32,000=132,000 records in all

$$\frac{33{,}000}{132{,}000} = 0.25 = 25\% \text{ desired records}$$

- The formula to determine the number of records to resample is

$$x = \frac{p(records) - rare}{1 - p}$$

where:

x is the required number of resampled recods

p is the desired proportion of rare values in the balanced data set,

records represents the number of records in the unbalanced data set, and

rare represents the current number of rare target values

Balancing the training data *(cont'd)*

- Set aside a number of non-fraudulent records
  - When resampling is not desired, a number of non-fraudulent records would be set aside instead
- To achieve a 25% balance proportion, we would retain only 3000 non-fraudulent records
- We would need to discard 96,000 out of 99,000 non-fraudulent records from the analysis
  - It would not be surprising if the model suffer out of starving for data in this way
- When choosing a desired balancing proportion, recall the rationale for doing so: in order to allow the model a sufficiently rich variety of records to lean how to classify the rarer value of the target variable across a range of situations
  - The balancing proportion can be lower if the analyst if confident that the rare target value is exposes to a rich variety of records
  - The balancing proportion should be higher if the analyst is not confident of this

Balancing the training data *(cont'd)*

- **The test data set should never be balanced**
  - The test data set represents new data that the model have not seen yet
  - Real work data is unbalanced, therefore, he test data set shouldn't be balanced either
  - All model evaluation will take place using the test data set, so that evaluative measures will be applied to unbalanced data

Establishing baseline performance

- Without a baseline, it is not possible to determine whether our results are any good
- Example: Suppose that we naively report that "only" 28.4% of the customers adopting the International Plan (see Table 3.3) will churn
  - This doesn't sound too bad, until we notice that the overall churn rate is only 14.49% (Figure 3.3)
  - This overall churn rate may be considered our *baseline*, against which any further results can be calibrated
  - Thus, belonging to the International Plan actually nearly doubles the churn rate – Not good!

Establishing baseline performance *(cont'd)*

- The type of baseline to use depends on the way the results are reported
  - For the churn example, we re interested in decreasing the overall churn rate
  - Thus, our objective should be to report a decrease in overall churn rate
- Example: Suppose the data mining model resulted in a predicted churn rate of 9.99%
  - This represents a 14.49%-9.99%=4.5% absolute decrease in the churn rate
  - But also a 4.5%/14.49%=31% in relative decrease in the churn rate
  - The analyst should make it clear for the client which comparison method is being used

Establishing baseline performance *(cont'd)*

- In an estimation task using a regression model, our baseline may take the form of a "$\bar{y}$ model"
  - $\bar{y}$ model - The model simply finds the mean of the response variable, and predicts that value for every record
- No data mining model should have a problem beating this $\bar{y}$ model
  - If your data mining model cannot outperform the $\bar{y}$ model, then something is clearly wrong
  - Recall that we measure the goodness of a regression model using the standard error of the estimate ***s***, along with the r^2
- A more challenging baseline would be using results already existing in the field
  - If current algorithm your company uses succeeds identifying 90% of all fraudulent transactions, then your model would be expected to outperform this 90%